Progressive Retrieval Attention: A Position Paper on Demand-Driven Context Expansion

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 7, 2026

Abstract

Large language models increasingly rely on longer context windows, retrieval-augmented prompts, and agentic search loops to access information beyond their parametric memory. These mechanisms are powerful, but they expose context to the model in a largely flat and eager form: tokens are either placed in the prompt, retrieved once before generation, or acquired through explicit tool calls outside the transformer's attention computation. This paper argues for a complementary direction: *Progressive Retrieval Attention* (PRA), a family of architectures and runtimes in which lightweight reference handles point to URI-addressed memory and can be expanded on demand into layer-specific attention memory. The central thesis is that long-context inference should move from monolithic token streams toward hierarchical, demand-driven context graphs. We distinguish naming from materialization, routing gists from transported token K/V, reference selection from chunk selection, and local language ability from reference-path adaptation. These separations position PRA as a bridge between sparse long-context transformers, retrieval-augmented generation, agentic progressive disclosure, and KV-cache runtime systems. We define the abstractions, propose a bounded recursive memory lifecycle and evaluation doctrine, and retain earlier standalone results as historical feasibility evidence. Those experiments predate the corrected chunk-routing architecture and cannot establish its causal efficacy. The program remains falsifiable by improved task quality per unit of context, latency, and memory.

1 Introduction

The transformer attention mechanism made it possible for models to condition every token on every earlier token in a sequence [17]. The same mechanism also created a central systems problem: when a model needs access to many documents, code files, logs, prior decisions, or external knowledge fragments, the simplest interface is to concatenate more text into a longer sequence. Modern long-context systems have stretched this interface dramatically, but a larger window does not by itself solve the problem of *selective use*. If only a small fraction of the context is relevant at any generation step, eager inclusion wastes computation and forces the model to perform implicit search inside an expensive dense attention substrate.

Retrieval-augmented generation (RAG) addresses part of this issue by selecting external passages before generation [11, 7, 8]. However, standard RAG often makes a single retrieval decision at the boundary of the model call. The retrieved passages are still inserted as flat prompt text, and the model must consume them whether or not each passage becomes useful later. Agentic systems take the opposite approach: they inspect tool descriptions, issue searches, open files, follow references, and progressively disclose detail only when needed [20, 15]. This behavior is effective

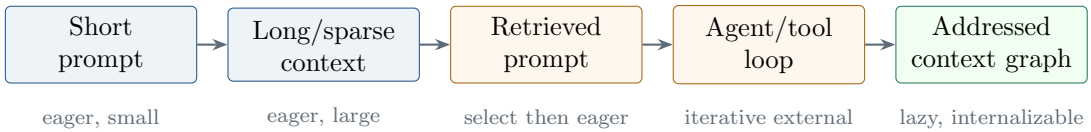

Figure 1: Context interfaces differ in when and where information becomes local to the model. PRA is a proposed design point, not a claim that the other interfaces should disappear.

but expensive because the loop is mediated by external actions, repeated model invocations, and hand-designed orchestration.

This paper proposes Progressive Retrieval Attention as a research program for internalizing part of that progressive-disclosure pattern. Consider a model diagnosing a failing authentication test in a large repository. Its local prompt can advertise compact handles such as `<REF_1>`, whose local reference table maps to `repo://app/auth#module`. The runtime resolves that object, partitions it into chunks, constructs small routing representations, and exposes detailed layer-specific memory only for selected chunks. If the module refers to a policy object or dependency, the same process can continue under a bounded expansion policy. The model need not read every advertised object eagerly; attention scores, learned gates, uncertainty, or explicit reference positions can trigger additional access as the diagnosis develops.

The central question is therefore: *can a model obtain better quality per unit of active context, latency, and memory by treating external information as a progressively materialized context graph rather than an eagerly flattened token sequence?* The paper develops the vocabulary needed to answer that question, separates architectural, runtime, optimization, and evaluation claims that are often collapsed under "retrieval," and states experiments that could reject the proposal.

1.1 Scope and Claim Types

The paper makes an architectural proposal, reports implementation facts, summarizes historical evidence, and advances hypotheses. The proposal is the separation among naming, routing, materialization, and transport. The current standalone implementation establishes that chunked, layer-specific caches and bounded child-first resolution can be constructed and consumed. Earlier experiments provide evidence about backbone compatibility and optimization controls, but their reference results used a superseded summary-level router. Claims about content-sensitive selection, cache reuse, recursive benefit, and end-to-end efficiency remain hypotheses. This distinction is maintained throughout the paper so that an implemented mechanism is not mistaken for a measured advantage.

Figure 1 locates the proposal among neighboring interfaces. The horizontal arrangement expresses increasing separation between the local token stream and the available information; it is not a historical sequence and does not imply that PRA should replace the earlier interfaces.

2 Motivation

2.1 The Cost of Flat Context

In a decoder-only transformer with dense self-attention, the attention matrix for a sequence of length T contains $O(T^2)$ pairwise interactions per layer. Efficient kernels and hardware-aware implementations reduce constants, and sparse-attention variants reduce asymptotic or effective

cost [1, 21, 9, 14]. Yet a conceptual mismatch remains: many tasks contain large amounts of potentially relevant context but only a small amount of immediately relevant evidence.

Consider a coding task. A repository may contain thousands of files. A developer rarely reads every file before making a change; they inspect a project map, open candidate modules, follow definitions, and read details near the call sites that matter. The useful information is embedded in a hierarchy: repository, package, file, class, method, block. The flat prompt interface collapses that hierarchy into a token sequence. Once collapsed, the model must recover structure through positional and lexical cues that were never designed as stable memory addresses.

The same pattern appears in scientific QA, technical support, legal analysis, and book-length reasoning. The system often knows that a reference exists before it knows whether the reference must be expanded. A long context window encourages eager expansion because the only available memory interface is token inclusion.

2.2 The Shallowness of One-Shot Retrieval

RAG retrieves documents or passages and places them in the model context [11]. This often improves factuality and makes model behavior auditable because generated answers can be traced to retrieved evidence. The limitation is that retrieval is usually performed at one granularity and one time. A retriever may return a passage that is too broad, too narrow, or missing the second-hop information required by the question. The generator then has limited ability to request a more specific subdocument without leaving the forward pass and entering an external loop.

Hierarchical retrieval partially addresses this issue by retrieving coarse summaries before detailed chunks, but the retrieved evidence is still usually materialized as prompt text. The generator receives the results of a pipeline rather than participating in a continuous memory-selection process. PRA asks whether part of this selection can be made differentiable, layer-local, or runtime-local.

2.3 The Expense of Agentic Search

Agentic systems use tools to search, read, execute, and revise. Their strength is not simply tool use; it is progressive disclosure. A capable agent first consumes compact descriptions and only loads detailed instructions, files, or logs when the task demands them. This style is naturally suited to large workspaces and multi-document tasks. Its weakness is cost and latency. Each search or file read may require a model turn, an external call, and a new prompt construction step.

The motivating question for PRA is therefore not whether agents should disappear. Rather, the question is whether the model/runtime boundary can absorb a lower-level version of progressive disclosure: reference-aware memory expansion during inference, with external agents reserved for high-level planning, verification, and action.

3 From Flat Context to Context Graphs

The repository example makes the required abstraction concrete. Before the model can route to the authentication implementation, the system needs a stable name for the object, an explicit relation to any child objects, and a representation compatible with the layer that will consume it. This section introduces those three pieces in that order. A useful way to keep them straight is to distinguish three levels of detail. The handle is an address, the routing gist is a small representation used to decide whether a chunk looks useful, and the token-level K/V is the evidence the decoder can actually read. Moving between these levels is the "progressive" part of PRA.

3.1 Reference Handles

A *reference handle* is a lightweight token or latent marker that names an external memory object without expanding it immediately. It plays the role of a typed pointer in the prompt: the pointer says where the object can be found, but it is not a summary of the object and does not consume the object's full token budget. In the simplest textual form:

`<REF_1>` $\rightarrow$ `repo://app/auth#module`

The handle is paired with a local reference table containing at least an identifier, token, and URI, with optional summary and metadata. The local table supplies the binding: `<REF_1>` has meaning only in the example or object that advertises it. Child edges exist only through this explicit table; URI-prefix similarity is not sufficient to create graph structure. The prompt can therefore advertise a possible source without pretending that the source has already been read.

3.2 URI-Addressed Memory

PRA assumes that external memory can be addressed by URIs. The scheme may denote local memory, files, repository objects, search results, database rows, or runtime objects:

`mem://manual#installation.cuda.vllm`

The anchor fragment after `#` is not merely a byte offset. It can encode semantic hierarchy: document, section, subsection, paragraph, function, class, or trace event. URI addressing gives the runtime a stable object to cache and gives the model a compact symbol for a potentially large subtree of context.

3.3 Recursive Anchors

Many references are best exposed recursively. A root object may advertise child anchors, and those child objects may advertise more specific anchors:

`mem://manual#root` $\rightarrow$ `mem://manual#installation` $\rightarrow$ `mem://manual#installation.cuda`

Recursive anchors turn context into a graph rather than a bag of chunks. Expansion must be child-first and bounded by shared depth, child, reference, token, latency, and byte budgets. Build states must prevent partially encoded cyclic entries from becoming searchable, while content/model/tokenizer/routing fingerprints protect cache identity. The model can begin with compact routing gists or optional summaries and progressively expand detail. This is particularly important for tasks where the correct evidence is not known from the initial query alone.

3.4 Layer-Specific Memory Cache

If an external fragment is encoded into memory for attention, the representation should respect the transformer's layer structure. A key/value pair computed for one layer is not generally interchangeable with another. In practical terms, PRA does not encode a chunk once and hand the same vector to every layer. It runs the chunk through the same model stack used by the prompt and captures the K/V representation appropriate to each PRA-enabled depth. PRA therefore uses a layer-specific memory cache:

$$\text{cache}[u, c, \ell] = (K_{u,c,\ell}, V_{u,c,\ell}, g^K_{u,c,\ell}), \tag{1}$$

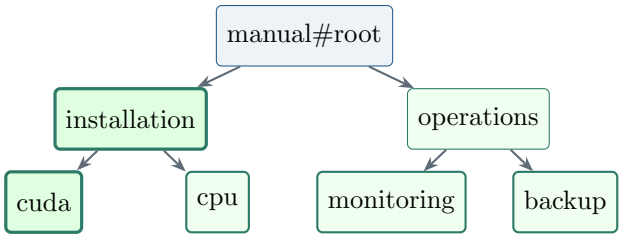

Figure 2: Recursive anchor expansion. A coarse handle exposes children; only the highlighted path is materialized to greater depth.

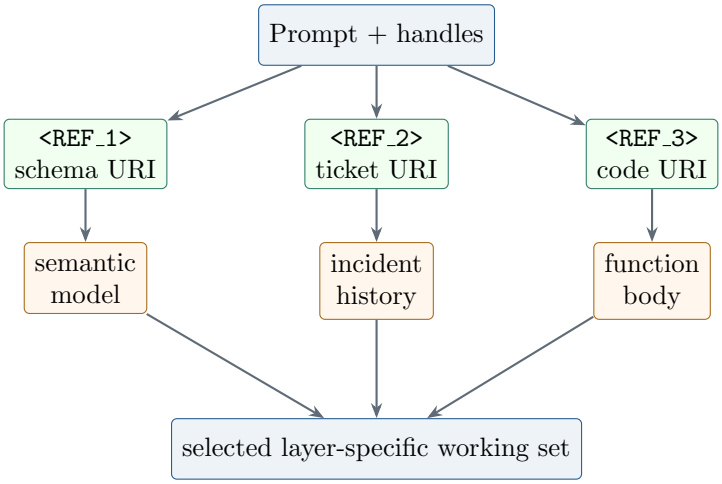

Figure 3: A context graph separates compact handles, addressable objects, and the active layer-specific working set.

where u is a URI, c a deterministic chunk, ℓ a transformer layer, (K, V) the full token memory, and g^K exactly one contextual routing gist derived from that layer's projected chunk keys. The gist is an index entry, not a compressed replacement for the chunk: it is cheap to compare with a query, while the full K/V preserves token-level detail after selection. This distinction matters because lower layers may encode lexical and syntactic information while higher layers may encode task-specific abstractions. Raw token embeddings or document summaries are not substitutes for layer-specific routing and attention memory.

4 Progressive Retrieval Attention

4.1 Conceptual Computation

Return to the authentication example. Local self-attention relates the test tokens and the code already present in the prompt. The memory branch answers a different question: given the current diagnosis, which advertised module chunks should become additional evidence at this layer? The computation below separates the cheap ranking operation from the more expensive attention over detailed evidence. Operationally there are two phases. A preparation phase resolves each advertised URI, partitions its content, and builds layer-specific chunk memories. A use phase lets the live prompt query the small chunk gists, materializes only the selected K/V, and fuses the resulting memory read with ordinary causal self-attention.

Let $X \in \mathbb{R}^{T \times d}$ contain the d-dimensional hidden states for T local tokens at layer ℓ. Standard causal self-attention first projects these states into queries Q, keys K, and values V. Each head has width d_h, and M masks future positions:

$$\mathrm{SelfAttn}(X) = \mathrm{softmax}\left(\frac{QK^\top + M}{\sqrt{d_h}}\right)V, \tag{2}$$

The output mixes values from preceding local positions according to query–key compatibility. A PRA layer preserves that path and adds memory selected from a cache. For routing, it uses the projected query at the most recent position, merges its heads into $q_{t,\ell}$, and compares it with one projected-key gist $g^K_{u,c,\ell}$ for chunk c of object u:

$$q_{t,\ell} = \mathrm{mergeheads}(XW_Q^{(\ell)})_t, \tag{3}$$

$$a_{u,c,\ell} = \cos(q_{t,\ell}, g^K_{u,c,\ell}). \tag{4}$$

The cosine score $a_{u,c,\ell}$ ranks a small routing representation rather than the full chunk. Hierarchical routing aggregates chunk scores to select at most k_r references, then independently selects at most k_c chunks inside each selected reference. Content-derived gists are the default. Summaries, when available, may replace, augment, or compete with content scores under an explicit policy, but missing summaries fall back to content. Let $\mathcal{A}_{t,\ell}$ be the selected reference–chunk pairs. This routing score answers "which drawers should be opened?"; it is not an attention weight over the evidence inside a drawer. A separate token-level cross-attention transport performs that read and computes

$$\mathrm{PRA}(X) = \mathrm{SelfAttn}(X) + \alpha \cdot \mathrm{softmax}\left(\frac{QK_{\mathcal{A},\ell}^\top}{\sqrt{d_h}}\right)V_{\mathcal{A},\ell}, \tag{5}$$

Here $K_{\mathcal{A},\ell}$ and $V_{\mathcal{A},\ell}$ concatenate full layer-ℓ token memory from selected chunks only, and α controls the memory residual. In the running example, the gist may select the password-validation chunk, but the decoder receives that chunk's token-level keys and values and can assign different attention weights to its individual tokens. Routing representations are therefore cheap selectors, not compressed substitutes for transported evidence.

The central research problem is selection: how should $\mathcal{A}_{t,\ell}$ be chosen? Possible triggers include contextual gist similarity, attention to explicit reference-token positions, uncertainty in the next-token distribution, learned gates, or runtime policies that trade accuracy against latency. Reference ranking, chunk ranking, and materialization must remain separately measurable.

4.2 PRA as Architecture, Runtime, and Research Program

The term PRA is intentionally broader than one neural module. As an *architecture*, it denotes attention layers that consume external reference memory through cross-attention, K/V injection, adapters, or another differentiable transport. As a *runtime*, it includes the resolvers, caches, URI stores, expansion policies, and schedules that decide when and how the memory is constructed. As a *research program*, it includes the benchmarks, ablations, scaling studies, and analyses needed to test demand-driven context expansion.

These scopes support different conclusions. A standalone prototype can validate cache construction and attention mechanics without showing readiness for a production-scale pretrained model. Runtime experiments can study scheduling and K/V reuse without settling how to train

the selector. Scaling studies can ask which task distributions benefit from context graphs without committing to one resolver or cache implementation. The following commitments keep those conclusions separable.

5 Conceptual Commitments

PRA is most useful as a research frame when several concerns remain separate. Collapsing them produces systems that may work but cannot be interpreted.

5.1 Naming Is Not Materialization

A reference handle states that a memory object exists and can be addressed. It does not imply that the object's text, embeddings, or K/V tensors have already been loaded. This distinction allows a runtime to expose a large logical context graph while maintaining a small active physical context. It also supports late binding: two deployments may resolve the same logical URI through different stores, permissions, or freshness policies.

The separation creates a useful sequence of representations:

$$\text{handle} \rightarrow \text{metadata} \rightarrow \text{optional summary} \rightarrow \text{text/chunks} \rightarrow \text{layer memory}. \tag{6}$$

Each transition has a cost and can be denied, cached, audited, or deferred. Progressive disclosure is therefore not merely truncation; it is movement through explicit levels of materialization.

For a reader following one reference end to end, the sequence is concrete. The prompt first carries a symbolic address. Resolution turns that address into an authenticated, versioned object. Encoding creates small layer-specific gists plus detailed token K/V. Routing scores the gists, and only then does memory attention consume selected K/V. A system may prepare some of these representations eagerly or retrieve them from a cache, but the logical boundaries remain the same.

5.2 Selection Is Not Transport

Selecting a URI and transporting its representation into a model are different operations. Selection may use lexical retrieval, vector similarity, explicit handle positions, learned gates, uncertainty, or an oracle. Transport may use prompt concatenation, cross-attention, adapters, or direct K/V injection. An experiment that changes both at once cannot determine whether a gain comes from finding better evidence or from presenting the same evidence through a better neural interface.

PRA evaluations should cross selection policies with transport mechanisms. For example, oracle selection with cross-attention tests the transport upper bound, while fixed retrieval with alternative transport tests the architectural path. Learned selection should be evaluated only after these simpler controls establish that selected memory can be used.

The same separation applies inside selection. Reference ranking answers which addressable objects remain candidates; chunk ranking answers which portions of those objects are materialized. Independent budgets are necessary to prevent a prolific reference from consuming every chunk slot. Reported cache size, selected gist count, and transported token count are different quantities and should never be used interchangeably.

5.3 Logical Isolation Must Survive Physical Batching

Each sequence has a logical memory address space even when kernels share physical tensors. Two rows may even use the same URI string for different content; the row-local reference table decides

what that name means. An efficient implementation can still run the prompt as one batch by keeping a private cache namespace for each row, routing row i only against namespace i, and masking padded memory positions. Correct batching may evaluate each row independently, pad one masked rectangle, or group rows into deterministic length buckets. These execution plans are equivalent only if empty rows receive an exact zero memory residual, outputs return to original order, and one row's cache cannot influence another row's logits. Padding allocation and bucket count are systems metrics, not semantic changes to routing.

5.4 Language Ability Is Not Reference Adaptation

Adding a memory path can improve a reference-formatted task while damaging ordinary language modeling. Conversely, freezing the local model can guarantee preservation while limiting adaptation. The relevant object is therefore a two-dimensional outcome: reference-task improvement and local-task retention. A useful method should report both, including the exact trainable parameter groups.

This distinction motivates staged optimization. First establish that each architecture learns ordinary language. Then initialize from the matching checkpoint and train only newly activated reference-specific modules. Finally, compare joint fine-tuning under a mixed plain/reference curriculum. Preservation obtained by freezing is a property of the optimization policy, not proof that an architecture is intrinsically immune to forgetting.

5.5 Memory Identity Requires Isolation

URI identity is part of the conditioning state. If a batched implementation combines references from unrelated examples without a corresponding mask, it changes the task and can leak evidence across samples. The standalone implementation exposed this issue directly: a batch-union cache made within-batch shuffling ineffective. Correctness required a cache per example. Efficient PRA runtimes must preserve the same logical isolation through cache indices or attention masks even when physical storage and kernels are shared.

6 Virtual Memory for Transformers

Virtual memory changed computing by separating the address space visible to a program from the finite physical memory resident at a given moment. Pages are named independently of their current location, loaded lazily, cached while useful, and evicted under pressure. Working-set theory connects recent access behavior to resource allocation rather than pretending that every address must remain resident [5]. Foundation-model context has an analogous tension: the logical information relevant to a task can be far larger than the token and K/V memory physically active during one decoding step.

PRA proposes a *virtual context space*. URIs and anchors play the role of stable logical addresses. Resolved text or structured objects are backing-store content. Encoded layer-specific K/V entries are resident pages. Selection or gating acts as an admission policy. A missing selected representation creates a context fault that triggers resolution and encoding. The active reference cache approximates a working set, while eviction and reuse determine whether the abstraction saves end-to-end cost.

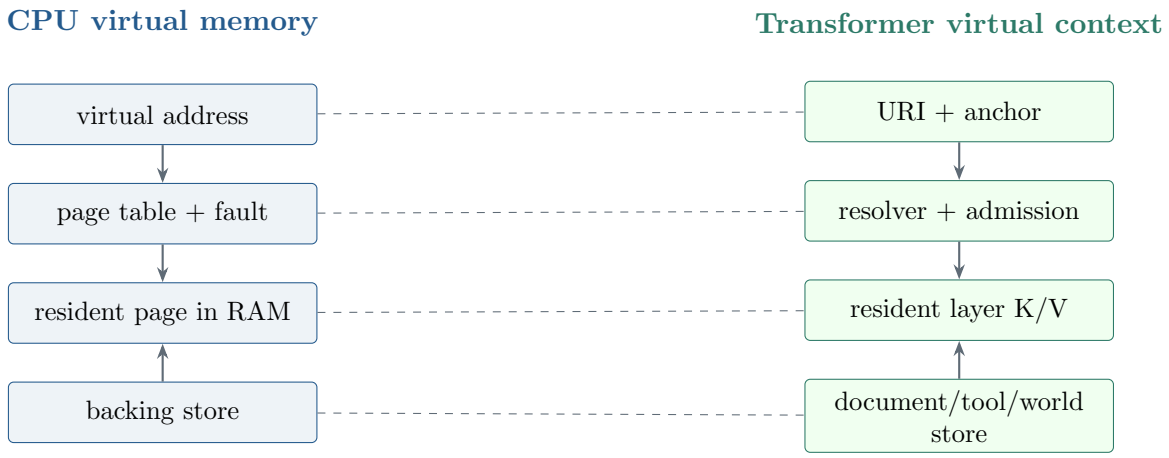

Figure 4: Virtual-memory analogy. Dashed lines indicate functional correspondences, not implementation equivalence.

6.1 Address Spaces, Protection, and Lifetime

A useful virtual context address is more than a string. It should identify scheme, authority, object, semantic anchor, version, tenant, and access policy. Two identical texts with different provenance may not be interchangeable; one URI whose content changes over time should not silently reuse stale K/V. Cache keys may therefore require a tuple such as

$$(u, v, \ell, m, \theta, p), \tag{7}$$

where u is URI, v content version, ℓ layer, m model identity, θ relevant encoder parameters, and p policy domain. Cache lifetime can range from one sequence to a user session, repository revision, model deployment, or immutable corpus version. Longer reuse improves amortization but increases freshness and revocation risk.

Permissions must be enforced before resolution and again when resident memory is shared. A model should not infer a protected reference because another tenant populated the same physical cache. Logical address spaces and attention masks must survive batching, deduplication, paging, and distributed placement.

6.2 Page Faults and Progressive Disclosure

A conventional page fault is generated by an exact address access. A model's "context fault" is less crisp: it may be inferred from explicit handle attention, uncertainty, a learned router, or a runtime policy. False negatives omit evidence; false positives waste resolution and encoding. This turns fault prediction into a joint learning-and-systems problem. Recursive anchors further resemble multi-level page tables only loosely: following a child changes semantic granularity rather than merely translating an address.

6.3 Where the Analogy Breaks

CPU pages are intended to preserve exact program semantics regardless of placement. Neural memory is approximate, content-dependent, and transformed by model parameters. Loading a document into K/V does not guarantee that the model will use it correctly. Attention can blend

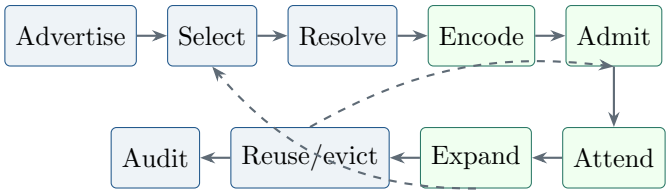

Figure 5: PRA lifecycle. Dashed feedback paths represent recursive selection and cache reuse.

untrusted entries, positional semantics may matter, and encoding may be more expensive than transport. The virtual-memory analogy is therefore valuable for identifying address, isolation, admission, working-set, and eviction problems; it is not evidence that operating-system policies can be copied unchanged.

7 A Reference-Memory Lifecycle

A PRA runtime can be described as a lifecycle rather than a single retrieval call:

1. **Advertise**: expose handles, summaries, types, provenance, and access policy.
2. **Select references**: rank candidate handles for a sequence, position, layer, or decoding interval.
3. **Resolve**: map selected URIs to immutable or versioned content.
4. **Expand**: recursively resolve explicit child tables under shared budgets and cycle policy.
5. **Chunk and encode**: transform content into bounded layer-compatible token K/V and one gist per chunk.
6. **Select chunks**: choose bounded materialization units inside retained references.
7. **Admit**: place entries in a bounded active cache under an admission policy.
8. **Attend**: allow authorized sequence positions and layers to consume the memory.
9. **Reuse or evict**: retain useful entries across tokens or requests, or release them under pressure.
10. **Audit**: record which version, URI, layer, and policy influenced the output.

This lifecycle suggests a cost objective. For task quality Q, prompt tokens T_p, encoded reference tokens T_r, latency L, and active cache bytes B, a system should be compared on a frontier such as

$$\max Q \quad \text{subject to} \quad c_p T_p + c_r T_r + c_l L + c_b B \leq C. \tag{8}$$

The constants depend on hardware, cache reuse, and service-level objectives. PRA is not successful merely because it shortens the visible prompt. Reference encoding that is repeated for every request may cost more than eager context. The economic case depends on selective admission, amortized reuse, or better quality under a fixed resource budget.

8 Architectural Placement

PRA need not occupy every transformer layer. Lower layers often build local lexical and syntactic representations, while later layers may provide more useful queries for semantic memory. A hybrid architecture can therefore place ordinary SelfAttention below PRA-enabled layers. The opposite hypothesis is also plausible: early access to external tokens may be necessary for composition across all depths. Full PRA, upper-layer PRA, mixed blocks, and sparse PRA placement should be treated as competing designs.

The standalone study compares four local layers, four PRA layers, and two local plus two PRA layers. In the three-seed follow-up, frozen-path reference loss is 4.7052 for full PRA and 4.7932 for hybrid PRA. Scratch-to-frozen improvement is also larger for full PRA (1.6493 versus 1.5783), so the tested hybrid does not benefit more from staging. Hybrid PRA does use half as many trainable memory output projections, but one tiny scale and an unlearned selector cannot resolve placement. A proper comparison still needs matched compute budgets, layer-wise reference attention diagnostics, and tasks stratified by whether required evidence is lexical, syntactic, or semantic.

9 Relationship to Existing Work

PRA draws mechanisms from several mature lines of work. The comparison below is organized by the boundary each line changes: interaction among local tokens, persistence of latent state, retrieval of external evidence, or acquisition through actions. This framing narrows the novelty claim to the combined access contract rather than any single component.

9.1 Long-Context and Sparse Transformers

Longformer, BigBird, Reformer, Routing Transformer, and related models reduce the cost of attending over long sequences by imposing locality, sparsity, hashing, routing, or structured global tokens [1, 21, 9, 14]. PRA is complementary. It does not primarily ask how to attend over a single long sequence more cheaply; it asks whether all candidate context should become part of that sequence in the first place.

9.2 Memory-Augmented Transformers

Compressive Transformers store compressed memories of past activations [13]. Memorizing Transformers retrieve nearest-neighbor memories from previous tokens [18]. RETRO retrieves chunks from a large text database and conditions generation on them [2]. PRA shares the ambition of memory-augmented generation but emphasizes explicit reference handles, URI addressing, recursive anchors, and layer-specific cache entries.

9.3 Hierarchical Attention and Adaptive Computation

Hierarchical Attention Networks demonstrated the value of multi-level document structure [19]. Universal Transformers and adaptive computation time study dynamic depth or recurrent refinement [4, 6]. PRA can be viewed as adaptive computation over context rather than only over layers or recurrent steps: the system spends more memory bandwidth on references that prove useful.

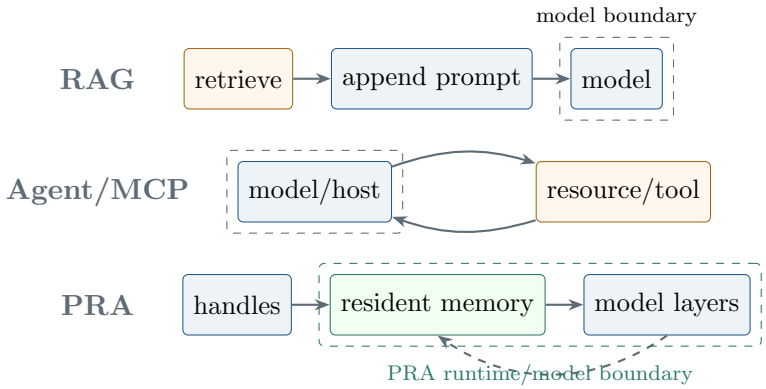

Figure 6: RAG selects before a model call, agents and MCP iterate across an application boundary, and PRA proposes addressable memory consumption within a runtime/model boundary.

9.4 Retrieval-Augmented Generation

RAG, REALM, and dense passage retrieval moved language models toward non-parametric memory [11, 7, 8]. PRA inherits their motivation but changes the interface between retrieval and generation. Instead of treating retrieval as a pre-generation pipeline stage, PRA treats retrieval products as addressable memory objects that can be expanded and attended to during inference.

9.5 Agentic Tool Use

ReAct and Toolformer show that language models can coordinate reasoning and external actions [20, 15]. PRA does not replace tool use. It attempts to make a subset of memory access cheaper and more structured by moving reference resolution and cache access closer to attention.

9.6 MCP and IDE Assistants

The Model Context Protocol (MCP) defines a client–server interface through which AI applications discover resources, prompts, and tools [12]. IDE assistants similarly expose repository search, symbol navigation, diagnostics, terminals, and version-control state. These systems make external capability discoverable, but the host application still decides how retrieved resource content enters a model context. PRA is complementary: MCP can supply the address space and resolver boundary, while PRA can define how selected resources become internal attention memory.

This combination suggests a layered interface. MCP or an IDE host handles authentication, capability discovery, transport, and side effects. A PRA runtime handles content identity, versioned encoding, cache admission, and model-local use. High-level tool calls should remain explicit and auditable; repeated read-only access to stable resources is the more plausible target for latent memory.

9.7 KV-Cache Runtime Systems

Serving systems such as vLLM's PagedAttention show that memory layout, paging, and cache management are first-class concerns for modern inference [10]. PRA extends this systems perspective from generated-token KV caches to external reference-memory KV caches. A production PRA runtime would need admission policies, eviction, batching, cache reuse, and memory isolation.

10 Embodied, Situated, and Referential Memory

Embodied and enactive approaches argue that cognition is shaped by recurrent interaction among brain, body, and environment rather than by isolated symbol manipulation alone [16]. The extended-mind thesis further asks when reliably available external artifacts participate in a cognitive process rather than merely serving as passive background [3]. PRA does not imply that a transformer is embodied or that neural attention reproduces human memory. These traditions instead offer a useful design question: what changes when external structure is not compressed entirely into internal parameters, but remains available through stable, action-conditioned references?

Human activity routinely relies on referential scaffolding. A person remembers that evidence is in a notebook, a clause is in a contract section, a procedure is in a manual, or an explanation belongs to a colleague. The remembered address, cue, and access routine can be more useful than a lossy internal copy. Retrieval is often situated: the object sought depends on the current task, physical or social environment, and sequence of prior actions. Hierarchical cues support progressive refinement from place or document to section and detail.

PRA translates only the computational aspects of this observation. Reference handles are cues; URIs stabilize identity; resolvers implement access routines; routing gists and optional summaries support selection; and layer-specific caches provide a temporary active representation. Recursive anchors make access path-dependent. The analogy predicts that a useful model memory system should preserve provenance and affordances, not only vector similarity. It also predicts failure when the environment is unreliable, addresses become stale, or access policy is ignored.

This perspective broadens evaluation beyond factual recall. A referential memory architecture should be tested on navigation, coordination, revision, and action: whether it can locate the right object, notice when it changed, follow a dependency, respect a permission, and update a plan after new evidence. Those are situated-memory behaviors, not merely next-token improvements.

11 A Future Foundation-Model Memory Stack

Foundation models are likely to require a hierarchy rather than one universal memory mechanism. Table 1 sketches distinct roles. The boundaries are conceptual and may be implemented by multiple coupled systems.

PRA is best viewed as an addressing and materialization fabric across this stack. It does not replace weights, local context, databases, tool protocols, or world models. It could allow compact working memory to refer to episodic events, semantic objects, and procedural capabilities; runtime policy would decide which representations can cross into model-local memory. Different classes require different consistency. Immutable literature can use content-addressed caches, while live operational state may require short leases and mandatory revalidation.

A complete stack also needs a control plane. It must assign identity, authenticate principals, describe provenance, negotiate capabilities, track versions, account for cost, and revoke access. The neural data plane then consumes authorized representations. MCP provides one possible application-level discovery and transport interface [12]; PRA addresses the separate question of selective internal materialization.

12 Enterprise and Professional Applications

Enterprise information is unusually compatible with context graphs because it already has identifiers, ownership, revisions, links, and permissions. The opportunity is not to pour every enterprise

Table 1: Conceptual foundation-model memory stack. Entries are architectural roles, not claims about current product implementations.

Layer	Typical content	Lifetime	Access mode
Parametric memory	linguistic and learned regularities	model version	dense forward pass
Working memory	current goals, dialogue, active evidence	sequence/session	local attention
Episodic memory	prior interactions and outcomes	user/project history	event retrieval
Semantic memory	documents, schemas, concepts, policies	corpus version	addressed retrieval
Procedural memory	tools, workflows, APIs, permissions	deployment	explicit action
World-state memory	environment, simulations, live systems	dynamic	observation/model update

object into a prompt, but to expose a governed address space whose active working set follows the task.

12.1 Software Engineering

A software task can begin from repository, branch, commit, package, symbol, test, issue, and build-log handles. GitHub objects and IDE symbol graphs provide stable navigation edges; PRA could keep interfaces and summaries active while materializing implementation bodies or historical diffs only when needed. Required controls include commit-aware cache keys, generated-file labeling, repository permission isolation, and tests that detect stale symbol memory. SWE-bench-style tasks and repository navigation benchmarks can measure whether progressive access reduces loaded tokens without hiding required dependencies.

12.2 Analytics, BI, and SQL

Data work spans warehouses, catalogs, SQL schemas, semantic layers, dashboards, lineage graphs, and business glossaries. A Snowflake table, BI metric, or transformation can be a versioned reference with ownership and freshness metadata. The model may first inspect a semantic metric definition, then expand the underlying model, columns, joins, and query history. Evaluation must penalize stale schemas, unauthorized columns, invalid SQL, and semantically wrong but executable queries. Live execution should remain an explicit tool action, while read-only schema memory is a candidate for caching.

12.3 Organizational Knowledge and Operations

Confluence pages, Jira issues, runbooks, incidents, and service catalogs form a cross-linked operational memory. Handles can preserve the distinction between approved policy, draft discussion, resolved incident, and current ticket state. Freshness and authority are central: an old runbook should not silently outrank a current service notice. Useful tasks include incident diagnosis, change-impact analysis, status reporting, and tracing a decision from ticket to documentation and code.

12.4 Scientific Literature

Scientific assistants need identity at article, version, section, figure, dataset, and claim level. Recursive anchors can move from abstract to method, result, appendix, and cited source. Provenance traces should distinguish quoted evidence from model inference. Benchmarks should test multi-paper synthesis, contradictory findings, retractions, and whether the model can expose the exact evidence path rather than only produce plausible prose.

12.5 Legal and Medical Assistance

Legal and medical domains make the risks clearest. Jurisdiction, effective date, patient identity, consent, and source authority are part of meaning, not optional metadata. PRA could help preserve these constraints through versioned, permissioned references and auditable access paths. It must not turn latent memory into an unreviewable decision channel. High-stakes outputs require authoritative-source validation, explicit uncertainty, human review, and strict separation between informational retrieval and actions affecting rights, treatment, or records. Evaluation should emphasize provenance completeness, stale-evidence detection, privacy leakage, and calibrated abstention rather than fluency.

13 Illustrative Design Space

Table 2 gives a conceptual comparison. The entries are hypotheses and design claims, not measured results.

Table 2: Conceptual comparison of context-access strategies. This table is illustrative and does not report experimental measurements.

Strategy	Context interface	Strength	Failure mode
Flat long context	Eager token sequence	Simple and general	High cost when relevance is sparse
One-shot RAG	Retrieved prompt passages	Good factual grounding	Retrieval granularity fixed before generation
Agentic search	External tool loop	Flexible progressive disclosure	High latency and orchestration cost
PRA	URI-addressed memory cache	Demand-driven latent expansion	Requires learned/runtime selection policy

14 Historical Feasibility Evidence

A standalone PyTorch pilot first provided limited implementation evidence for parts of this position. At 524,288 processed tokens and seeds 1 and 7, mean plain WikiText-2 losses were 4.7691 for SelfAttention, 4.7457 for full PRAttention, and 4.7574 for a two-SelfAttention/two-PRAttention hybrid. Its version-1 reference task suggested a memory-path contribution, but regenerated data per model seed and labeled all candidate parts as relevant. More importantly, the old implementation routed one raw summary embedding per reference, collapsed a batch query in an early path,

and materialized whole references. We therefore retain all reference-conditioned measurements as engineering history rather than use them for a causal claim about the corrected architecture.

A controlled follow-up used paired seeds 1, 7, and 21; a 1,048,576-token budget; a fixed tokenizer; a fixed 512-example reference corpus and split; and SelfAttention controls parameter matched through FFN width. Table 3 summarizes ordinary-language results. Because external memory is absent in this phase, these values remain relevant to backbone compatibility. Full PRA remains close to its matched control, while hybrid PRA does not outperform its matched control. This supports architectural compatibility, not superiority.

Table 3: Three-seed ordinary-language follow-up. Test cross-entropy; lower is better.

Architecture	Parameters	Test loss
SelfAttention, same width	4,216,320	$4.6378 \pm .0380$
Full PRA	4,479,488	$4.5463 \pm .0270$
Hybrid PRA	4,347,904	$4.6049 \pm .0359$
SelfAttention, matched to full	4,478,976	$4.5706 \pm .0037$
SelfAttention, matched to hybrid	4,347,648	$4.5725 \pm .0108$

The reference follow-up compares training from scratch, frozen-reference-path adaptation, and direct joint fine-tuning. For full PRA, respective valid-reference losses are 6.3545, 4.7052, and 4.9504; for hybrid PRA they are 6.3715, 4.7932, and 4.9883. Pretraining therefore improves final loss, and the frozen substage is best under the tested budget. It also preserves plain WikiText loss exactly. Direct joint tuning increases plain loss by 0.1598 for full PRA and 0.1429 for hybrid PRA, so it is not sufficient here. Full PRA benefits at least as much from staging as the hybrid. Doubling full-PRA ordinary-language pretraining from the pilot budget lowers mean loss by 0.1994, indicating that it benefits from a longer tested schedule without establishing how long is enough.

The historical causal reference result was negative. On the fixed task, selected-reference top-1 accuracy remained near the 0.4202 test chance rate. After frozen adaptation, full-PRA loss was 4.7052 with valid references, 4.7116 with shuffled references, 4.7140 with irrelevant references, and 4.7051 with oracle content. Hybrid differences were similarly small. The branch contributed relative to complete disabling, but this contribution was not demonstrably tied to correct content. These values diagnose the old summary-similarity selector and optimization regime; they neither validate nor falsify the corrected per-layer chunk router until the experiment is rerun.

A post-refactor smoke rerun now exercises the corrected batched router at fixed total split counts 2 and 5. Twenty tiny CUDA runs share 32 source entries, one BPE tokenizer, context length 128, a fixed data-generation and split seed, and an eight-step budget while pairing model seeds {1, 7, 21, 42, 87} across architecture and split count. For full PRA, mean valid-reference loss improves over disabled-reference loss by 0.0189 ± 0.0030 at split 2 and 0.0312 ± 0.0048 at split 5. Every seed has a positive gap, establishing reproducible branch activation. Mean shuffled-reference changes are only 0.0002 ± 0.0006 and -0.0006 ± 0.0009, however, and split-5 reference top-1 is 0.150 ± 0.102 against a 0.25 uniform-candidate rate. Correct content is therefore not causally established. SelfAttention is exactly invariant to the reference interventions, as required of the control. Because split count also changes the target span and processed tokens, these values remain end-to-end regression and cost diagnostics rather than an isolated split-count effect.

These measurements do not establish the central efficiency thesis. Reference encoding was performed separately for every example on a small GPU, selective triggering was disabled, and cache reuse was not measured. They instead show why the conceptual separation matters: naming and a reference-specific residual can be engineered while selection can still fail independently. The

corrected implementation adds the instrumentation and controls needed for a cleaner test, but software completion is not empirical confirmation.

15 Evaluation Doctrine

PRA requires counterfactual evaluation because a single task score cannot reveal how memory was used. At minimum, a promising checkpoint should be evaluated with valid references, a disabled reference path, shuffled references, irrelevant references, empty references, oracle references, and all evidence visible in local context. Targets and prompt structure must remain fixed across these conditions. Shuffling must occur across examples while preserving reference count and approximate length, and batched caches must prevent cross-example leakage.

Four families of outcomes should be reported together:

1. **Task quality**: loss, perplexity, calibrated accuracy, or task-specific utility.
2. **Reference and chunk causality**: valid-versus-disabled, valid-versus-shuffled, valid-versus-irrelevant, oracle-chunk, full-reference, and gist-only deltas.
3. **Preservation**: ordinary language and non-reference task quality before and after adaptation.
4. **Resource use**: processed tokens, available versus selected memory tokens, synchronized cache/routing/attention latency, bucket padding ratio, throughput, peak memory, cache bytes, and reuse rate.

Evidence of genuine reference use requires more than a disabled-path gap. A branch can act as a generic domain adapter regardless of supplied content. Valid references should outperform shuffled and irrelevant content, ideally by larger margins on examples proven to be locally insufficient. Reference routing and chunk routing should separately rank known relevant objects and evidence spans above controls. Claims should be repeated across seeds and reported with matched token, parameter, and compute budgets.

16 Research Agenda

The PRA research program should proceed through increasingly demanding stages.

Standalone controlled models. Train small decoder-only transformers where all memory objects, reference handles, chunks, cache entries, and evaluation labels are controlled. The previous three-seed study is complete only for the superseded router. The corrected content-gist hierarchy must first rerun paired valid, disabled, shuffled, irrelevant, empty, oracle-chunk, full-reference, and gist-only conditions with both URI and chunk/span labels. Only then should a trainable router with an auxiliary relevance objective be compared, followed by five-seed replication, distance bins, mixed plain/reference curricula, optional-summary modes, and recursive-depth tasks. Scaling to the Small model is deferred until valid content reliably beats shuffled and irrelevant controls.

Pretrained-model integration. Adapt pretrained Hugging Face models using cross-attention adapters before attempting direct KV injection. This stage should isolate compatibility issues such as rotary position encodings, grouped-query attention, cache layout, and fine-tuning stability.

Runtime and serving integration. Study how reference memories are resolved, encoded, cached, paged, shared, and evicted. The central metric is no longer only accuracy, but accuracy per token, per millisecond, and per byte of cache.

Scaling and theory. Develop task families where relevant context occupies a controllable fraction of the available corpus. PRA should be evaluated against full-context, sparse-attention, RAG, summary-first RAG, and agentic-search baselines.

Selection and gating. Compare projected content gists, explicit reference-token attention, optional summary evidence, learned routers, uncertainty triggers, and hybrid policies while holding selected-chunk transport fixed. The old summary-similarity selector did not improve with language pretraining and remained near chance despite lower final task loss. Reference and chunk selection should be evaluated through hit/recall/precision/F1, MRR, NDCG, span IoU, counterfactual task loss, and learning speed, not only whether an expected URI entered a cache.

Hierarchical expansion. Construct benchmarks in which a root object advertises candidate child anchors but the answer requires one or more deeper fragments. Optional summaries can be introduced as a controlled routing condition rather than assumed metadata. Measure expansion depth, branching factor, missed-evidence rate, and quality per resolved token. Flat single-hop tasks cannot test the progressive part of PRA.

Training and preservation. Compare isolated reference-path adaptation, joint fine-tuning, adapters, low-rank updates, and curriculum mixtures. Report both reference improvement and local-task retention. Determine whether zero-initialized memory residuals remain stable at larger scale and whether learned gates avoid permanent saturation.

Safety, provenance, and access control. Treat reference resolution as a privileged operation. Study signed or versioned URIs, tenant isolation, policy-aware cache keys, provenance traces, revocation, poisoning detection, and whether the model can infer protected content through shared memory representations.

16.1 Falsification Criteria

The research program should be abandoned or substantially revised if repeated, well-controlled studies show any of the following:

- valid references do not outperform shuffled or irrelevant references once formatting and capacity are controlled;
- equivalent prompt concatenation or RAG matches quality at lower end-to-end cost;
- reference encoding and cache management erase any savings from shorter local context, even with reuse;
- learned expansion policies fail to outperform simple oracle-budgeted retrieval or cannot maintain recall;
- local language ability degrades unacceptably under the adaptation needed to use memory;

- URI and cache isolation cannot be enforced under realistic batching and multi-tenant serving.

These criteria make PRA a testable systems-and-model hypothesis rather than a label that can absorb any retrieval mechanism after the fact.

17 Risks and Limitations

PRA introduces several risks. First, reference selection can fail silently: if the system does not expand the right anchor, the model may answer from incomplete context. Second, URI-addressed memory creates provenance and security questions. A reference table is a powerful interface; poisoning or misaddressing entries can mislead the model. Third, cache construction may move cost rather than eliminate it. Encoding references separately is only beneficial when cache reuse, selective expansion, or reduced prompt length outweighs encoding overhead. Fourth, pretrained integration is technically difficult because external K/V must respect layer semantics, position encodings, attention variants, and batching constraints.

Fifth, content-independent branch effects can be mistaken for retrieval. A memory residual may improve a domain simply because it contributes additional activations, even when references are shuffled. Sixth, batching can violate semantic isolation: a physically shared cache must not expose one sequence's references to another, and empty rows must not inherit projection bias. Seventh, preservation claims can be artifacts of freezing rather than architecture. Eighth, hierarchical expansion can amplify latency variance and create denial-of-service surfaces when references recursively fan out. Ninth, raw summaries can introduce a representation mismatch when compared with projected layer queries; optional summary evidence must inhabit the same contextual layer space. Finally, stale cache entries create a consistency problem: URI, content version, model/tokenizer/routing fingerprints, and encoded layer memory must remain aligned.

These limitations motivate careful experimental design. PRA should not be judged by a single accuracy number. It should be evaluated through controlled ablations that measure answer accuracy, reference and chunk ranking, expansion depth and budget exhaustion, available and materialized token counts, full-context token count, timing decomposition, padding allocation, cache hit ratio, content-specific counterfactual deltas, plain-task retention, and cache isolation. The historical three-seed follow-up addresses capacity, training order, preservation, and first-order counterfactuals for version 1; corrected routing, recursive behavior, and end-to-end efficiency remain untested empirically.

18 Conclusion

Progressive Retrieval Attention proposes a shift in how foundation models interact with memory. Rather than treating context as a flat token budget to be filled eagerly, PRA treats references as first-class computational objects that can be named, selected, materialized, attended to, expanded, reused, revoked, and audited. The model's logical context may then be much larger than its active working set, just as a program's address space can exceed resident memory. Unlike conventional paging, however, neural materialization is approximate and learned; success must be demonstrated, not presumed.

The long-term vision is a foundation-model memory stack with explicit boundaries among parametric knowledge, working context, episodic history, semantic resources, procedures and tools, and changing world state. URI identity, provenance, freshness, hierarchy, permissions, and cache lifetime become architectural concerns rather than application-side annotations. Protocols such

as MCP can expose resources and actions, agents can manage high-level plans, and PRA-like runtimes can test whether repeated read-oriented access can move closer to attention without losing governance.

The historical standalone follow-up supports backbone compatibility and demonstrates useful preservation controls, but it does not establish causal selection or the efficiency thesis for the corrected architecture. The implementation now makes the proposal sharper: projected per-layer content gists route independently to references and chunks; only selected full-token K/V is transported; recursion is explicit, bounded, and child-first; and physical batching preserves logical memory isolation. These are testable commitments, not results. The program should proceed only if corrected and eventually learned selectors make valid chunks consistently beat shuffled and irrelevant controls, local capabilities remain preserved under matched adaptation, and end-to-end quality improves per unit of latency, encoded tokens, and active memory against strong long-context, RAG, RETRO, and agentic baselines. If those conditions fail, the virtual-context proposal should be revised or rejected. If they hold across scales and domains, foundation models may evolve from systems with larger prompts into systems with principled memory architectures.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[3] Andy Clark and David J. Chalmers. The extended mind. *Analysis*, 58(1):7–19, 1998.

[4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[5] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[6] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[7] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.

[8] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

[9] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large

language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[12] Model Context Protocol Contributors. Model context protocol specification. `https://modelcontextprotocol.io/specification/2025-06-18/`, 2025. Protocol revision 2025-06-18.

[13] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, 2021.

[15] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[16] Francisco J. Varela, Evan Thompson, and Eleanor Rosch. *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press, 1991.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[18] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[19] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of NAACL-HLT*, 2016.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[21] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.